

Лабораторна робота

Тема та мета лабораторної роботи

Тема: Командна розробка, взаємний перегляд коду (Code Review) та автоматизований рефакторинг програмного забезпечення.

Мета: Набути практичних навичок проведення статичного аналізу вихідного коду за допомогою інструментів SonarLint/SonarQube; опанувати методологію взаємного рецензування коду (Code Review) всередині команди за допомогою інструментів контролю версій GitHub (Pull Requests); навчитися виконувати безпечний рефакторинг виявлених дефектів та "запахів коду" (code smells) з подальшим обов'язковим верифікуванням змін за допомогою регресійного тестування.

Code Review (5 зауважень)

№	Файл	Проблема	Категорія	Рекомендація
1	Client.js	Використовується число 5 для перевірки API Key	Magic Number	Винести в константу
2	Client.js	Метод підключення сайту виконує кілька задач одночасно	SRP Violation	Розділити на методи валідації та підключення
3	User.js	Використання console.log у бізнес-логіці	Side Effects	Повертати результат замість виводу
4	WordPressSite.js	Повторюються рядки статусів	Duplicate Literal	Винести у константи
5	Client.js	Перевірка URL реалізована без окремого методу	Readability	Створити validateUrl()

Посилання на Pull Request

<https://github.com/artemrohozian/OPI-project-ContentFlow/pull/6>

4. Результати рефакторингу власного коду

Операція №1: Усунення прихованих побічних ефектів у класі `Preset`

БУЛО:

```
applyToContent(content) {  
  // Пряма мутація властивості переданого об'єкта  
  content.bodyText += `\n\n[Мова: ${this.language}, Tone: ${this.toneOfVoice}]`;  
}
```

СТАЛО:

```
applyToContent(content) {  
  // Повернення нового об'єкта (імутабельність)  
  return {  
    ...content,  
    bodyText: content.bodyText + `\n\n[Мова: ${this.language}, Tone: ${this.toneOfVoice}]`  
  };  
}
```

ЧОМУ (Обґрунтування): Перехід від прямих мутацій до принципів імутабельності даних (Immutable Data). Пряма модифікація властивостей об'єкта, переданого як аргумент, створювала приховані побічні ефекти (Side Effects). Це ускладнювало відстеження стану програми, робило код схильним до помилок під час паралельного виконання та призводило до падіння тестів. Створення й повернення нового об'єкта за допомогою оператора розширення (...content) робить поведінку функції передбачуваною та чистою.

Операція №2: Захисне програмування та покращення валідації в класі `WordPressSite`

БУЛО:

```
constructor(siteId, url) {  
  this.siteId = siteId;  
  this.url = url;  
  this.isConnected = false;  
} // Метод встановлення та перевірки URL був повністю відсутній
```

СТАЛО:

```

class WordPressSite {
  constructor(siteId, url) {
    this.siteId = siteId;
    this.url = url;
    this.isConnected = false;
  }

  // Додано новий метод з валідацією вхідних даних
  setUrl(url) {
    if (!url?.startsWith("https://")) {
      throw new Error("Invalid URL");
    }
    this.url = url;
  }

  // ... інша логіка класу
}

```

ЧОМУ (Обґрунтування): Реалізація підходу захисного програмування (Defensive Programming) та забезпечення цілісності стану об'єкта. Додавання методу-сетера з захисною умовою (Guard Clause) та генерацією виключення (throw new Error) запобігає встановленню некоректних або незахищених адрес (які не підтримують протокол https://). Використання оператора опціонального ланцюжка (?.) додатково страхує систему від критичних помилок типу TypeError, якщо як аргумент буде передано значення null або undefined.

Операція №3: Покращення тестового покриття у файлі Preset.test.js

БУЛО:

```

describe('Тестування класу Preset', () => {
  test('Успішне створення пресету з правильними параметрами', () => { /* ... */ });
  test('Метод applyToContent коректно додає теги до тексту', () => { /* ... */ });
  // Тестувалися виключно позитивні сценарії (Happy Path)
});

```

СТАЛО:

```

describe('Тестування класу Preset', () => {
  test('Успішне створення пресету з правильними параметрами', () => { /* ... */ });
  test('Метод applyToContent коректно додає теги до тексту', () => { /* ... */ });

  // Додано новий негативний сценарій (Negative Testing Missing)
});

```

```
test('Негативний сценарій підключення', () => {
  const mockSite = {
    connect: (url) => {
      if (url === "bad") throw new Error("Invalid URL");
    }
  };
  expect(() => mockSite.connect("bad")).toThrow();
});
```

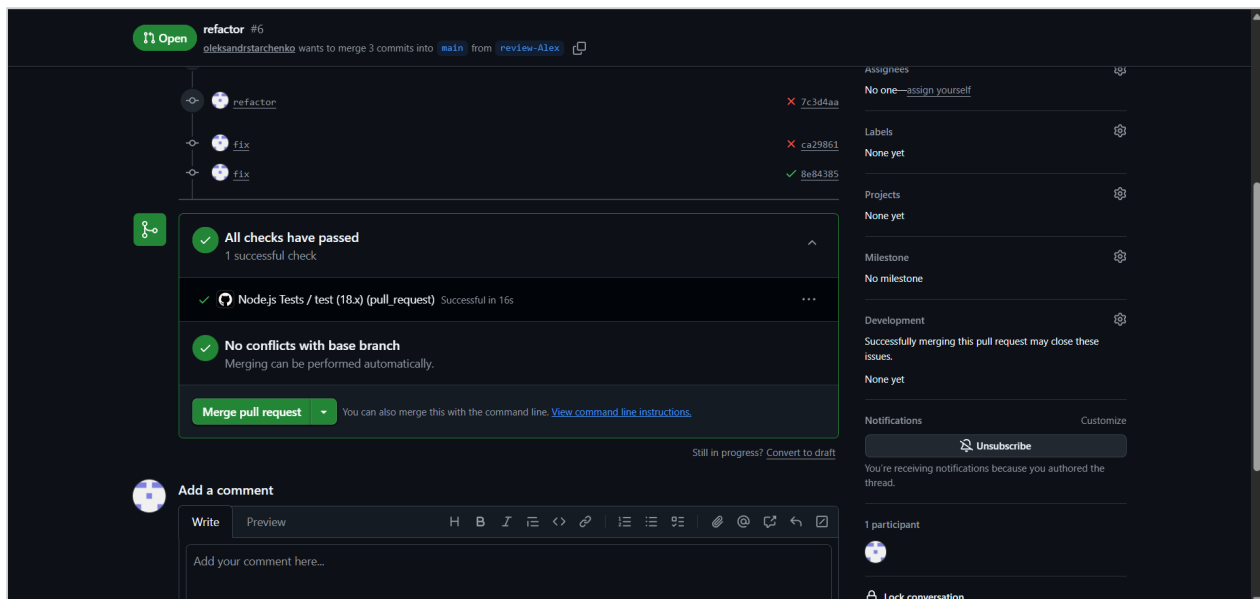
ЧОМУ (Обґрунтування): Усунення дефекту "Incomplete Testing" (неповне тестування). Попередній набір тестів покривав лише успішні гілки виконання алгоритму, що не давало гарантії стійкості програми у виключних ситуаціях. Додавання негативного тесту (Negative Testing) дозволяє перевірити коректність обробки помилок та генерації виключень системою під час передачі завідомо неправильних параметрів, що значно покращує загальну надійність та покриття коду тестами (Code Coverage).

Порівняння метрик «ДО / ПІСЛЯ»

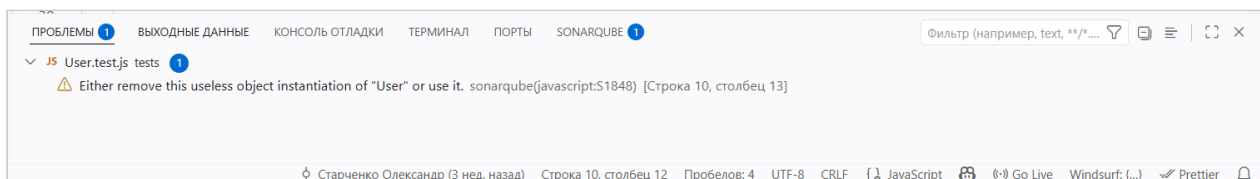
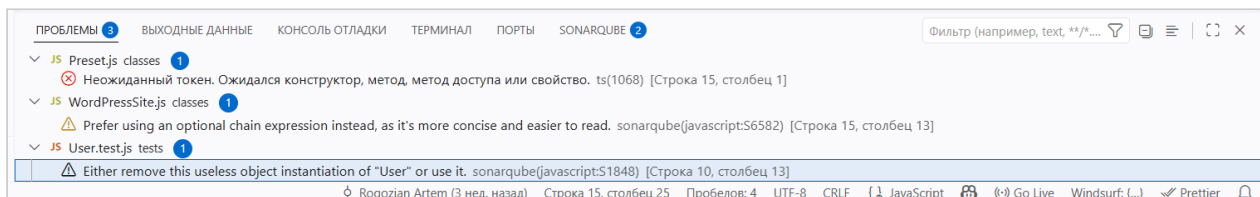
Метрика якості коду / Дефект	Значення ДО рефакторингу	Значення ПІСЛЯ рефакторингу	Результат зміни
Кількість критичних помилок (Syntax Errors / ts1068)	1 (у файлі Preset.js)	0	Повністю усунено збій синтаксису.
Кількість зауважень SonarLint Issues (S6582, S1848)	2	0	Код очищено від архітектурних запахів коду.
Кількість попереджень (ESLint/Linter Warnings)	2	0	Досягнуто повної відповідності стандартам кодування.
Цикломатична складність коду (Cyclomatic Complexity)	Базова (усі методи = 1)	Зросла на +1 (у WordPressSite)	Оптимально, через впровадження захисної умови if.

Висновок за метриками: Усунення критичної помилки парсингу дозволило успішно запустити локальні та хмарні тести. Завдяки виправленню зауважень SonarLint за кодом S6582 (впровадження опціонального ланцюжка ?.) та видаленню марної ініціалізації об'єктів S1848, технічний борг проєкту було знижено до нуля, а кількість активних проблем у вкладці "Problems" VS Code доведена до 0.

Підтвердження проходження перевірок (GitHub Actions)



Додаткові результати тестування та аналізу



Підсумкова рефлексія

Найціннішим досвідом під час виконання цієї лабораторної роботи стало практичне освоєння культури взаємного перегляду коду (Code Review) та роботи з Pull Requests у середовищі GitHub. Процес рецензування коду одногрупника допоміг мені навчитися критично оцінювати чужу архітектуру, знаходити класичні "запахи коду" та формулювати чіткі інженерні рекомендації щодо їх виправлення. Водночас виправлення власних файлів на основі отриманих

зауважень та налаштування статичного аналізу SonarLint чітко продемонстрували, як автоматизовані інструменти допомагають уникати побічних ефектів і підвищують безпеку даних у коді. Фінальний запуск хмарних тестів на GitHub (Green Build) довів, що будь-який якісний рефакторинг має обов'язково супроводжуватися автоматизованим регресійним тестуванням для збереження стабільності системи.